

Engineering RAG Assistant

Retrieval-Augmented Generation over documents, Bitbucket code and Jira

Project: advanced-rag-gcp | Status as of 2 October 2026

What this system is

A locally hosted assistant that answers engineering questions from three kinds of knowledge: uploaded documents (PDF, DOCX, TXT, Markdown), source code in Bitbucket repositories, and Jira issues. Every answer is generated only from retrieved chunks of the sources the user selected, and cites the file path, document or Jira key it came from.

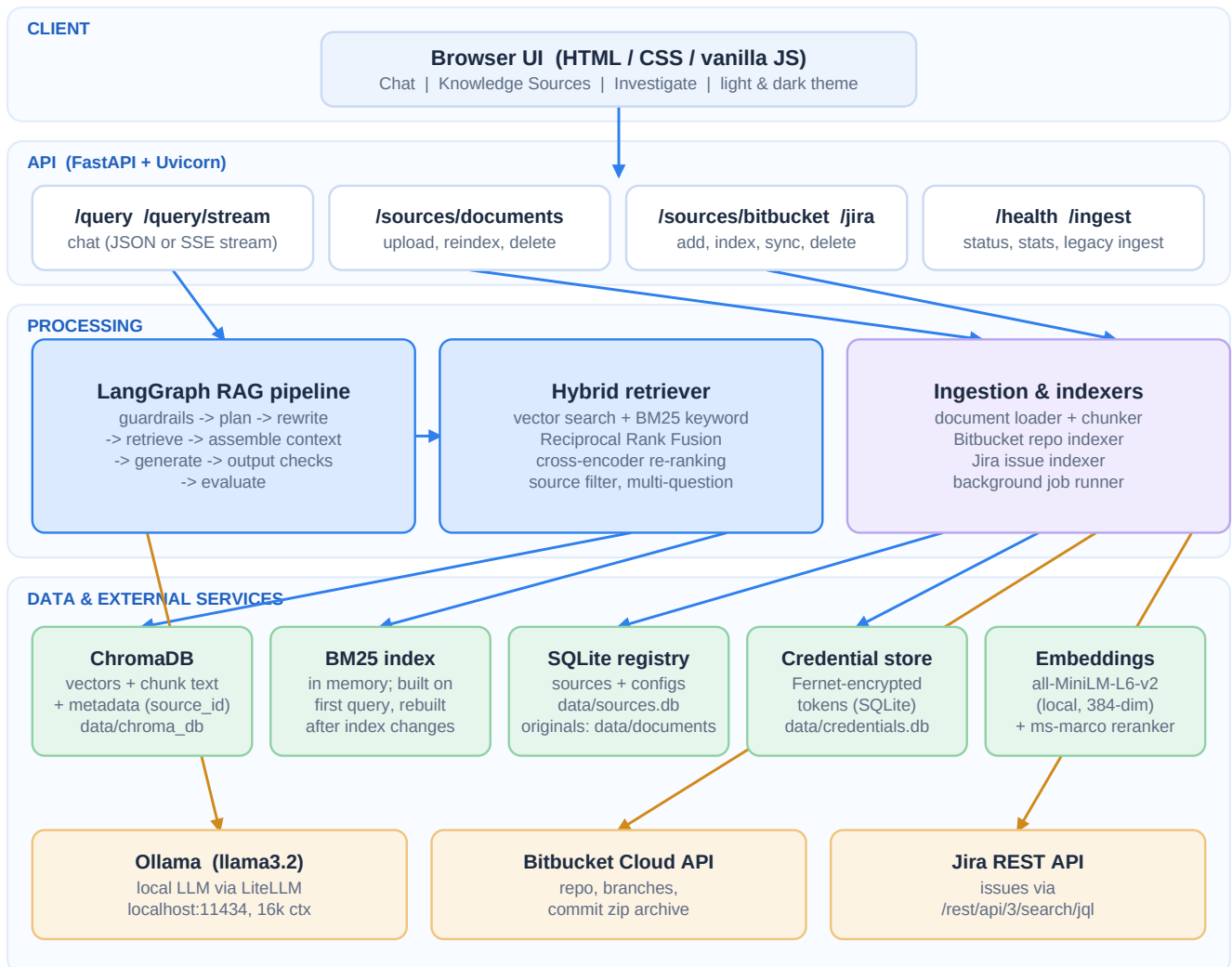
At a glance	
Runs as	One FastAPI server (backend + web UI) on http://localhost:8000, plus a local Ollama model server
Orchestration	LangGraph state machine with 8 processing nodes and a blocked-input branch
Retrieval	Hybrid: ChromaDB vector search + BM25 keyword search, fused with RRF, re-ranked by a cross-encoder
Language model	Ollama llama3.2 (3B) through the LiteLLM gateway; Gemini / OpenAI supported by configuration
Knowledge sources	Documents, Bitbucket Cloud repositories, Jira Cloud / Server projects
Storage	ChromaDB (vectors), SQLite (source registry, encrypted credentials), local disk (original files)
Cloud path	Docker image, Terraform for Google Cloud Run, GCS, BigQuery, Secret Manager, Vertex AI

Contents

1.	System architecture
2.	Technology stack
3.	Query pipeline (LangGraph)
4.	Hybrid retrieval
5.	Knowledge sources and ingestion
6.	Data storage and security
7.	Frontend
8.	API reference
9.	Configuration
10.	Running locally and deployment
11.	Current status and limitations

1. System architecture

The application is a single Python process. FastAPI serves the static web UI and the REST API; the LangGraph pipeline answers questions; indexers turn sources into searchable chunks. All data stays on the machine except calls to Bitbucket and Jira (and to a cloud LLM, if one is configured).



Two flows run through this system:

- **Indexing (write path)** - a document upload, or an Index / Sync on a Bitbucket repository or Jira project, loads the content, splits it into chunks, embeds each chunk, and stores it in ChromaDB with metadata that records which source it belongs to. The keyword index is rebuilt afterwards.
- **Answering (read path)** - a chat question runs through the LangGraph pipeline: safety checks, retrieval restricted to the selected sources, prompt assembly, generation by the LLM, output checks and evaluation. The UI streams the answer and shows the sources and the chunks that were used.

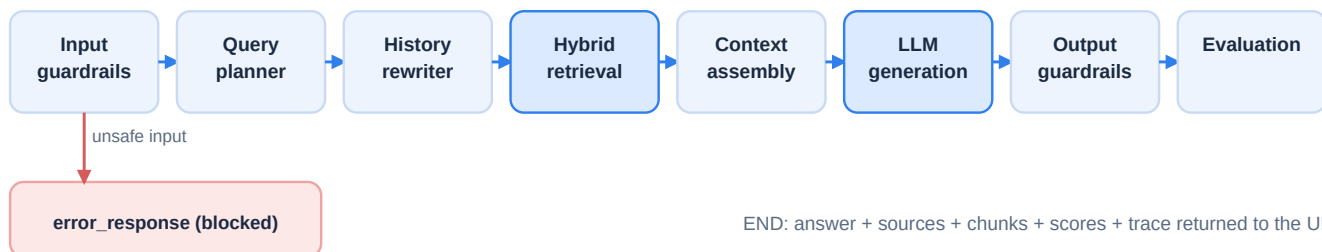
2. Technology stack

Versions are those installed in the project environment (.venv-macos, Python 3.12.13).

Layer	Technology	Version	What it is used for here
Web server	FastAPI + Uvicorn	0.111.0 / 0.30.1	REST API, Server-Sent Events streaming, and serving the UI from /static
Validation	Pydantic, pydantic-settings	2.8.2 / 2.3.4	Request/response models; typed settings loaded from .env
Orchestration	LangGraph	0.2.14	The RAG pipeline as a state machine (nodes, edges, conditional branch)
RAG toolkit	LangChain (core / community)	0.2.16 (core 0.2.38)	Document loaders, text splitters, Chroma vector-store wrapper, Document objects
LLM gateway	LiteLLM	1.60.0	One API for Ollama, Gemini and OpenAI; retries (tenacity), budget guard, streaming
LLM runtime	Ollama + llama3.2 (3B)	0.35.0	Local text generation on the Mac GPU; context window raised to 16,384 tokens
Vector database	ChromaDB	1.5.9	Persistent vector store with metadata filtering (data/chroma_db)
Embeddings	sentence-transformers: all-MiniLM-L6-v2	3.0.1	384-dimension embeddings computed locally; reads the first 256 tokens of a chunk
Re-ranking	Cross-encoder ms-marco-MiniLM-L-6-v2	-	Scores each (question, chunk) pair to separate relevant from noisy chunks
Keyword search	rank-bm25 (BM25Okapi)	0.2.2	Exact-term matching; complements vector search
Document parsing	pypdf, docx2txt, python-docx	4.3.1 / 0.9 / 1.1.2	Text extraction from PDF and Word
Source APIs	requests	2.34.2	Bitbucket Cloud REST + archive download; Jira REST search
Guardrails	Regex rules; Microsoft Presidio	2.2.354	Prompt-injection, length and empty-input checks; optional PII redaction (off in .env)
Evaluation	RAGAS	0.1.21	Faithfulness / relevancy / precision scores per answer (see section 11)
Security	cryptography (Fernet)	50.0.2	Encrypts Bitbucket and Jira tokens at rest
Metadata store	SQLite (Python stdlib)	-	Source registry and encrypted credential tables
Observability	structlog; Langfuse (optional)	24.4.0 / 2.36.1	Structured JSON logs; LLM tracing when Langfuse keys are configured
Frontend	HTML, CSS, vanilla JavaScript	-	No build step or framework; light and dark themes
Testing	pytest, FastAPI TestClient	8.2.2	78 offline tests (fake embeddings, mocked APIs)
Packaging / cloud	Docker, docker-compose, Terraform	-	Container image (Python 3.11-slim), local compose, Google Cloud Run infrastructure

3. Query pipeline (LangGraph)

Each question creates a RAGState dictionary that flows through the graph. Every node reads what it needs from the state and writes its results back. If the input guardrails block the question, the graph branches to an error response instead.

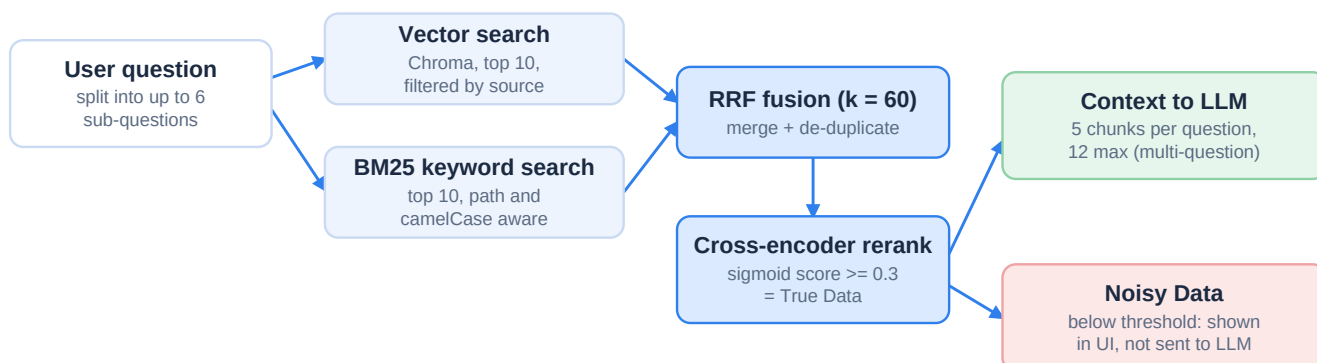


Node	What it does	How
Input guardrails	Rejects unsafe or invalid questions	Regex prompt-injection patterns, 4,000-character limit, empty check; optional Presidio PII redaction
Query planner	Classifies intent	Keyword rules: factual, analytical (compare / vs), conversational (earlier / you said), summary
History rewriter	Makes follow-up questions self-contained	Only for conversational intent: the LLM rewrites the question using the last 3 turns
Hybrid retrieval	Finds the relevant chunks	See section 4. Honours the source filter chosen under Search In
Context assembly	Builds the prompt context	Numbers each chunk and labels its type and source: [3] (code) repo/lib/instagram/publish.ts
LLM generation	Writes the answer	System prompt: answer only from sources, one heading per question, cite file paths / Jira keys, never invent code. Temperature 0.1, max 1,500 output tokens, last 2 turns of history
Output guardrails	Checks the answer	Groundedness score (overlap of 3-word phrases with the context), refusal and harmful-content checks
Evaluation	Scores answer quality	RAGAS metrics when available; results shown in the RAG Internals panel

Streaming endpoint. The UI uses POST `/api/v1/query/stream` by default. It runs the input guardrails and the same hybrid retrieval, builds the same prompt, then streams tokens from the LLM as Server-Sent Events, followed by a metadata event with the cited sources. The non-streaming `/query` endpoint runs the full graph, including output guardrails and evaluation.

4. Hybrid retrieval

Retrieval quality decides answer quality. The retriever combines meaning-based and word-based search, then re-ranks the merged candidates with a more accurate model.



Step	Details	Setting
1. Question splitting	A message with several questions (one per line or ending in '?') is split; each sub-question is retrieved separately so every topic gets its own chunks	up to 6 sub-questions
2. Vector search	Embeds the question with all-MiniLM-L6-v2 and finds the nearest chunks in ChromaDB; the source filter is applied inside the Chroma query	retrieval_top_k = 10
3. BM25 search	Keyword scoring over all chunks. The tokenizer splits paths and camelCase, so 'lib/instagram/publish.ts' and 'publishToInstagram' both match 'instagram'	top 10
4. Reciprocal Rank Fusion	score = sum of $1 / (60 + \text{rank})$ across both lists; chunks found by both rise to the top; duplicates are merged by chunk_id	k = 60
5. Cross-encoder re-rank	Scores each (question, chunk) pair jointly; sigmoid of the logit gives 0-1	threshold = 0.3
6. Selection	Chunks at or above the threshold are True Data (sent to the LLM); the rest are Noisy Data (shown in the UI only)	5 per question; 12 max

Source filtering

The chat sidebar's **Search In** list sends the selected sources with each question. Registered sources are matched by source_id; older uploads (made before document management existed) have no source_id and are matched by file name. When every box is ticked, no filter is sent and all sources are searched. A failing filter raises an error rather than silently searching everything.

```

{"question": "How does publishing work?",
"source_ids": ["bb-9a4c0450"],
"legacy_files": []}

```

5. Knowledge sources and ingestion

All three source types end the same way: chunks with metadata are written to ChromaDB in one atomic replace (new chunks are added before old ones are removed, so a failed re-index keeps the old content), then the BM25 index is rebuilt.

Documents

- **Upload:** PDF, DOCX, UTF-8 TXT and Markdown, up to 50 MB each, from Knowledge Sources - Documents.
- **Registration:** each upload gets an ID (doc- . . .) in the SQLite registry, and the original file is kept in data/documents/ under that ID (never the user's file name), which makes **Reindex** possible.
- **Chunking:** RecursiveCharacterTextSplitter, 512 characters with 128 overlap; each chunk keeps its page number.
- **Delete** removes the chunks, the keyword entries, the original file and the registry row. **Older uploads** (before this feature) can be listed with a text preview and removed, but not re-indexed.

Bitbucket repositories

- **Connect:** workspace, repository, branch, and an account email + API token. The token is tested against the Bitbucket API, then stored encrypted.
- **Index:** resolves the branch to its latest commit (falling back to the repository's default branch if the configured one does not exist), downloads that commit as a single zip archive, and reads it in memory.
- **File selection:** source and text files up to 400 KB (about 60 extensions plus Dockerfile, Makefile, README...). Skips build and dependency folders (node_modules, dist, build, vendor, .venv...), lock files, minified files, binaries, and likely secrets (.env, *.pem, *.key, id_rsa, .npmrc). Limit 5,000 files.
- **Chunking:** 900 characters with 100 overlap, split preferably at class / function boundaries, so each chunk fits the embedding model's 256-token window. Every chunk starts with Repository: ... File: path, and a file-tree overview is indexed for questions about the whole repository.
- **Sync:** re-indexes only if the branch has a new commit. Jobs run in a background thread; the UI polls every 3 seconds and shows indexing, ready or a readable error.

Jira projects

- **Connect:** https base URL, project key (validated, e.g. BT), account email and API token.
- **Index:** pages through /rest/api/3/search/jql (Jira Cloud), falling back to /rest/api/2/search for Jira Server / Data Center. Up to 5,000 issues.
- **Content per issue:** key, summary, type, status, priority, resolution, assignee, reporter, dates, parent, labels, components, fix versions, link, description and the latest 30 comments. Jira's rich-text format (ADF) is converted to plain text.
- **Chunking:** 900 characters with 100 overlap; later chunks repeat the issue key and summary so each can be cited. Sync re-fetches all issues.

Chunk metadata (stored with every chunk in ChromaDB):

Field	Example	Purpose
source_id	bb-9a4c0450, doc-e31c..., jira-1a2b...	Filtering, deletion and replacement per source
source_type	document bitbucket jira	Labels the chunk in the prompt and the UI
source_file	amitkushwah/Dhanvi-Social-Automation/lib/ai/prompts.ts	Citation shown under each answer
chunk_id	bb-9a4c0450:lib/ai/prompts.ts:3	De-duplication in rank fusion
extra fields	file_path, branch, commit, language, url, issue_key, status, page_number	Context and links back to the original

6. Data storage and security

Store	Location	Contents
ChromaDB	data/chroma_db/	Collection rag_documents: chunk text, 384-d vectors, metadata
BM25 index	process memory	Built from ChromaDB on the first question after startup; rebuilt after every index change
Source registry	data/sources.db (SQLite)	Tables sources, document_configs, bitbucket_configs, jira_configs
Credentials	data/credentials.db + data/.credential_key	Fernet-encrypted tokens, referenced by credential_id
Original documents	data/documents/	Uploaded files named by generated source ID, kept for re-indexing
Model caches	~/.cache/huggingface, Ollama store	Embedding, re-ranker and llama3.2 weights

Security measures

- **Tokens never leave the server in plain text.** Bitbucket and Jira tokens are encrypted with Fernet (AES-128-CBC + HMAC-SHA256) before being written; API responses only report `credential_configured: true`. Tokens are decrypted only inside the indexers.
- **Secret files are never indexed** from repositories (.env, private keys, keystores, package-manager credentials).
- **Safe file handling.** Uploaded files are stored under generated IDs, never under user-supplied names, and paths are checked to stay inside data/documents.
- **Input validation.** Jira project keys are validated before being placed in a JQL query; Jira URLs must use https; uploads are type- and size-checked.
- **Grounded answers.** The prompt restricts the model to the retrieved sources and the output guardrail scores groundedness.
- **Production path.** The credential store sits behind an interface designed to be swapped for Google Secret Manager, which the Terraform configuration already provisions.

7. Frontend

Three files in `ui/` (`index.html`, `style.css`, `app.js`), served by FastAPI. No framework and no build step.

Area	Features
Chat tab	New chat button; Search In list with type toggles (Documents, Bitbucket, Jira) and one box per source; streaming answers with clickable source chips; Enter to send, Shift+Enter for a new line
RAG Internals panel	Pipeline trace with timings, evaluation scores, True Data and Noisy Data chunks with scores
Knowledge Sources tab	Documents (upload, status, chunk count, Reindex, Delete, Older uploads with Remove); Bitbucket and Jira (add with connection test, Index, Sync, Delete, last commit / sync, errors)
Investigate tab	Placeholder for a later phase
Themes	White / light-blue (default) and dark; toggled from the header and remembered in the browser
Progress	Polls source status every 3 s while indexing and shows a notification when it finishes or fails

8. API reference

All endpoints are under `/api/v1`. Interactive documentation is at `http://localhost:8000/docs`.

Method	Path	Purpose
POST	<code>/query</code>	Ask a question; full pipeline; JSON answer with chunks, scores and trace
POST	<code>/query/stream</code>	Ask a question; answer streamed as Server-Sent Events
GET / POST	<code>/sources/documents</code>	List documents / upload one (multipart)
GET / DELETE	<code>/sources/documents/{id}</code>	Get or delete a document and its chunks
POST	<code>/sources/documents/{id}/reindex</code>	Re-index a document from its retained original
GET / DELETE	<code>/sources/documents/legacy</code>	List older uploads / remove one (<code>?source_file=...</code>)
POST	<code>/sources/bitbucket/test</code>	Test Bitbucket credentials without saving them
GET / POST	<code>/sources/bitbucket</code>	List repositories / add one
GET / PUT / DELETE	<code>/sources/bitbucket/{id}</code>	Get, update or delete a repository source
POST	<code>/sources/bitbucket/{id}/index /sync</code>	Start a background full index / sync if new commits
POST	<code>/sources/jira/test</code>	Test Jira credentials without saving them
GET / POST	<code>/sources/jira</code>	List projects / add one
GET / PUT / DELETE	<code>/sources/jira/{id}</code>	Get, update or delete a Jira source
POST	<code>/sources/jira/{id}/index /sync</code>	Start a background index / refresh of all issues
GET	<code>/health/live /health/ready /health/stats</code>	Liveness, readiness and system statistics
POST / GET	<code>/ingest/file, /ingest/files, /ingest/gcs, /ingest/stats</code>	Legacy ingestion and statistics

9. Configuration

Settings come from `.env` through `pydantic-settings (src/config.py)`. The values that shape behaviour today:

Setting	Current value	Effect
OLLAMA_MODEL	llama3.2	Local model used for generation (when no Gemini / OpenAI key is set)
OLLAMA_NUM_CTX	16384 (default)	Context window requested from Ollama
GEMINI_API_KEY / OPENAI_API_KEY	empty	Setting one switches generation to that provider via LiteLLM
EMBEDDING_MODEL	sentence-transformers/all-MiniLM-L6-v2	Embedding model (384 dimensions)
VECTOR_STORE_TYPE	chroma	chroma locally; vertexai for Vertex AI Vector Search
CHROMA_PERSIST_DIR	./data/chroma_db	Where vectors are stored
RERANKER_MODEL	cross-encoder/ms-marco-MiniLM-L-6-v2	Re-ranking model
RERANKER_THRESHOLD	0.3	Minimum score for a chunk to reach the LLM
retrieval_top_k / final_top_k	10 / 5	Candidates per retriever / chunks per question
ENABLE_INPUT_GUARDRAILS / OUTPUT	true / true	Safety checks on questions and answers
ENABLE_PII_DETECTION	false	Presidio PII redaction of questions
ENABLE_INLINE_EVALS / EVAL_SAMPLE_RATE	true / 1.0	Evaluate every answer
HF_HUB_OFFLINE	1	Use cached Hugging Face models only (no downloads at runtime)
LITELLM_BUDGET_LIMIT	10.0 (USD)	Stops paid LLM calls once spending reaches the limit

10. Running locally and deployment

Local (macOS)

Run from the project folder in two terminals. FastAPI serves both the API and the UI, so there is no separate frontend server.

```
# Terminal 1 - local model service (skip if Ollama is already running)
ollama serve

# Terminal 2 - backend + UI on http://localhost:8000
.venv-macos/bin/python -m uvicorn src.api.main:app --host 127.0.0.1 --port 8000

# Optional: auto-reload during development
.venv-macos/bin/python -m uvicorn src.api.main:app --reload --port 8000

# Tests
.venv-macos/bin/python -m pytest -q
```

Containers

- **docker/Dockerfile:** multi-stage build on python:3.11-slim (builder installs dependencies; runtime image copies them).
- **docker-compose.yml:** rag-api on port 8000 and an optional standalone chromadb service on port 8001.

Google Cloud (Terraform, [infra/terraform/main.tf](#))

Resource	Role
Cloud Run v2 service	Runs the API container; scales from 0 to 10 instances
Service account + IAM roles	Identity for the service to reach the resources below
Cloud Storage bucket	Document storage; ingestion via /ingest/gcs
Artifact Registry	Hosts the container image
BigQuery dataset + table	Evaluation metrics (rag_metrics)
Secret Manager secrets	LLM and Langfuse keys; target for the credential store
Vertex AI (configurable)	Gemini models and Vector Search as managed replacements for Ollama and ChromaDB

11. Current status and limitations

Area	Status
Documents	Done: upload, registry, reindex, delete, older-upload removal
Bitbucket	Done: connect, background index, commit-aware sync. Indexed: Dhanvi-Social-Automation, 196 files, 1,467 chunks
Jira	Done in code and tested against simulated responses; not yet tested against a live Jira project
Source filtering	Done for both chat endpoints
Repository / branch / project discovery dropdowns	Planned (endpoints return not_implemented)
Investigate tab	Planned

Known limitations

- **Answer accuracy is limited by the 3B model.** llama3.2 sometimes misnames files or mixes in details from a different chunk. A larger code model (for example qwen2.5-coder:7b, about 4.7 GB) is the main recommended upgrade.
- **Embedding model is small.** all-MiniLM-L6-v2 is general-purpose and reads 256 tokens; a code-aware embedding model with a longer window would improve retrieval, at the cost of re-embedding all sources.
- **RAGAS evaluation currently fails** (the installed version expects an additional field), so answers show no evaluation scores until it is fixed.
- **Keyword index is rebuilt in full** after each index change and kept in memory - fine for thousands of chunks, not for very large corpora.
- **Index jobs run in-process.** A server restart interrupts them; such sources are marked as errors at startup and can be indexed again.
- **Older uploads** cannot be re-indexed because their originals were not kept; re-upload them to manage them fully.

Recommended next steps

#	Step	Benefit
1	Switch to a larger local code model	More accurate, better-cited answers about code
2	Upgrade the embedding model	Better retrieval of code and long chunks
3	Fix the RAGAS integration	Per-answer quality scores in the UI and BigQuery
4	Connect and verify a live Jira project	Confirms the Jira path end to end
5	Repository / project discovery and the Investigate tab	Completes the remaining planned phases